

A Full Metro-scale Time-Dependent Traffic Route Model

Hamid Shafieasl
University of Utah
Salt Lake City, Utah, USA

Xiaoyue Cathy Liu
University of Utah
Salt Lake City, Utah, USA

Jeff M. Phillips
University of Utah
Salt Lake City, Utah, USA

ABSTRACT

We describe how to train an urban traffic model on a full metropolitan road network, up to 5-minute resolution on a week-long model. The training data is from GPS probes representing about 5% of all traffic in that area over a 30-day period. Building such a large model (about 100,000 links, and 2,016 time windows for a week), presents many challenges. First, the 5% of all traffic (about 2.5 million trips) is still incredibly sparse on this scale. This means it is essential to smooth out the data and error evaluation, and also not learn too many parameters at risk of over-fitting. Second, because the scale is so large, to be able to both train parameters and also generate feasible routes and not just static distributions, it is important we learn a PageRank-style model. We demonstrate the model's ability for route forecasting around three changes in traffic behavior.

CCS CONCEPTS

• **Computing methodologies** → **Modeling and simulation**; Spatial and physical reasoning; • **Information systems** → *Geographic information systems*.

KEYWORDS

PageRank, road traffic, GPS trajectory, map matching, hidden Markov model, graph diffusion, urban mobility, per-link traffic estimation

ACM Reference Format:

Hamid Shafieasl, Xiaoyue Cathy Liu, and Jeff M. Phillips. 2026. A Full Metro-scale Time-Dependent Traffic Route Model. In *Proceedings of 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26)*. ACM, New York, NY, USA, 22 pages. <https://doi.org/XXXXXX.XXXXXXX>

1 INTRODUCTION

A full metro-scale traffic model, we posit, should contain the following components. (1) It should consider traffic on each link of a road network. Otherwise, it is not compatible with low-level route planning, actual map-matched GPS-traces. (2) It should be temporal, with a small resolution like 5 minutes, and spanning a week. Traffic patterns are variable at this resolution level, due to commute and meal regularity. The week long models are necessary for regular changes in behavior over that period. (3) It should be able to efficiently produce actual road-network routes mirroring GPS traces, starting in each time window. These are necessary for producing

realistic routes that can adapt to the environment for planning and forecasting. The efficiency is necessary so this can be deployed in other AI infrastructure for generative modeling. (4) Finally, it should learn from and be trained on actual data so it can represent the true human behaviors and adapt if new data arises.

In this paper we demonstrate how to build such a system on the greater Salt Lake City metropolitan area. This represents a cohesive and connected ecosystem with about 100,000 links. We train this model on GPS tracing representing about 5% of the traffic for the area for a 30 day period (about 2.5 million routes). The scale of the data is large, but manageable by today's standards – the real challenges is the sparsity. After filtering and cleaning, only about 180 million GPS points remain; but at a 5-minute resolution with 2,016 time periods over a week, there are over 800 million links×time-windows to model. Even if we had a longer observation period or a higher percentage of routes, a similar challenge would remain since many routes are concentrated on busy thoroughfares and are peak times of the day. The tail of density is extremely long.

We manage to effectively build a model satisfying this desiderata through several innovations. Our baseline is a PageRank model that treats directed network links as graph vertices, and intersections model outgoing edges. We then categorize links by speed and number of lanes, and use data to learn transition probabilities between edges that favor transitions to or from higher throughput roads. However, routes typically move towards thoroughfares as they depart, and then move away from them as they reach their destination; so we design a two-phase PageRank that has separate transition probabilities as it gears up, and then speeds down. Finally, when a route ends, PageRank “teleports” to a random node to start the next walk; however, this does not reflect the pattern of route origins. So we use our observations of route starting points to also inform this teleportation probability, akin to trust-rank models []. We demonstrate that each of these steps is necessary to build an effective learned model.

Another challenge is in a trustworthy evaluation. Due to the data sparsity, even without a cross-validation split, it is not enough to directly predict the link density accuracy. Across the 4 full weeks, only 3% of links×time have at least one observation in a week, and only 1% have at least one observation in more than one week. So we disperse the sparse observations we have using graph diffusion [19]. We show how to apply this in a controlled way so that for each full week of observation, each link×time has non-zero measure, and the average relative change in value is less than the median number of difference in observations among the links×time which have observations in more than two weeks. That is, this diffusion makes the observations dense in a natural way that does not disrupt the data below the noise floor. From here, we can evaluate the model by training on one week, and measuring on the next.

Ultimately, we demonstrate an efficient, fine-grain, and full model on a metro scale data set. It produces natural temporally encoded

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGSPATIAL '26, November 3–6, 2026, Riverside, CA, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/11

<https://doi.org/XXXXXX.XXXXXXX>

routes, and its error is below the week-to-week variability of the diffused observations. Moreover, we demonstrate on two scenarios (a large sporting event, and an accident-induced lane closure) how we can use it to effectively forecast heavy traffic and investigate what-if scenarios.

2 RELATED WORK

This paper designs a method to fit a calibrated model of vehicle traffic at the resolution of *every single road link* of an entire city ($\sim 100,000$ links) at 5-minute temporal resolution over a week, using probe trajectories on a fixed road network. The closest prior work falls into three groups: agent-based traffic simulators that take demand as an input rather than fit it from data [21, 22]; origin-destination estimators that operate at a coarser zone or node resolution [11, 12]; and PageRank-style random walks applied to urban networks for centrality or coarse congestion prediction [8, 23, 24].

The closest comparable effort in scope is MATSim [21, 22], a long-running agent-based simulator that, given a synthetic population, schedules, and a network, performs *forward* simulation of per-link traffic at every time of day. Our problem is the *inverse*: given observed probe trajectories, infer a per-link popularity field and validate it against held-out weeks. The two efforts complement rather than compete; we use the MATSim network model of Salt Lake City as our graph, but the calibration-from-data step is outside MATSim’s scope.

A long line of transportation work estimates city-wide origin-destination (OD) matrices from cellular records combined with link counts [11] or from sparse probe trajectories combined with link counts [12]. These methods produce an OD matrix at zone or node resolution: each entry quantifies how many trips begin in zone a and end in zone b , but commits to no particular route in between. Our learned per-link traffic field is finer modeling unit than OD estimation, and also incorporates temporal modeling.

PageRank has been adapted to urban networks in a variety of ways complementary to ours. It has been used for structural centrality ranking of links in the network [8], or other ranking of urban spaces [24]. It has been used to model congestion, for instance by Wang *et.al.* [23] on a coarse 4,000 cell grid of Beijing. These methods predominantly use PageRank for analysis, and may even evaluate it against some observations, but it is not trained on full traffic observations, and not at the level of link or time granularity that we build. Our higher-resolution model, fit to real observations, will provide a better setting with which to perform these sorts of analysis.

3 DATA AND PREPROCESSING

3.1 Probe corpus

Our raw input is a corpus of sparse GPS trajectory records collected by Intrix **JEFF: get details from Cathy** in the Salt Lake City metropolitan area between August 31, 2018 and September 30, 2018, a continuous 30-day observation window. The data are distributed as a set of data rows, with ones belonging to the same device are interleaved chronologically. Each row corresponds to a single GPS fix and contains four fields:

Column	Meaning
did	anonymized device identifier (string)
ts	timestamp, formatted YYYY-MM-DD HH:MM:SS
lat	latitude of the GPS fix (decimal degrees)
lon	longitude of the GPS fix (decimal degrees)

The raw corpus contains 924,379,163 GPS locations; however roughly 80% of them fall outside the bounding box around the Salt Lake City metro area we study (see Figure 1), and are dropped at the first preprocessing stage, leaving 179 million locations inside the metro area. After full preprocessing and the map matching of Section 3.2, the corpus is reduced to approximately 2,477,000 distinct trips, of which about 714,594 survive map matching and contribute to the population matrix. The remaining trips are discarded for one of three reasons: fewer than five GPS locations, locations that fall outside the road network, or no viable Viterbi path (from map matching). At this stage the data are purely geographic point clouds: the observations are not yet associated with any road link, and no network structure has been introduced.

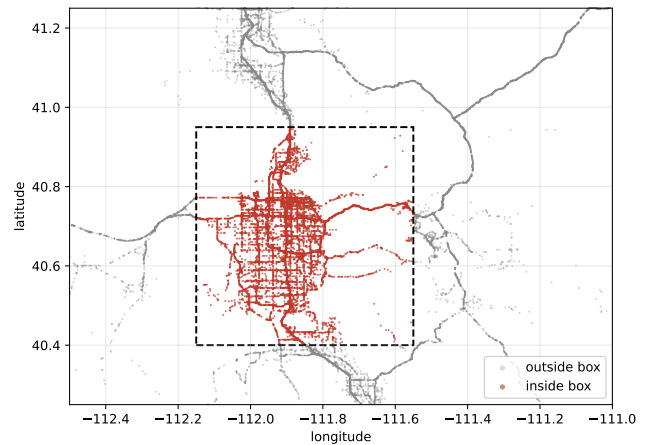


Figure 1: Raw GPS locations (random sample of $\sim 10^5$ from 924 M) and the SLC bounding box. Red points fall inside the box ($40.4\text{--}40.95^\circ\text{N}$, $112.15\text{--}111.55^\circ\text{W}$, dashed); gray fall outside.

3.2 Map-matching

We project the raw GPS fixes onto the road network with the Leuven implementation [1, 2] of the hidden Markov map-matcher of Newson and Krumm [3]. The matcher models the GPS-to-link perpendicular distance as a zero-mean Gaussian with standard deviation σ (in metres); σ is the only parameter and is corpus-dependent. Newson and Krumm fit $\sigma \approx 4.07$ m on their Seattle corpus; on our sparser SLC corpus we refit to be self-consistent. We re-run the matcher across a range of candidate σ (with a candidate pruning radius $\text{max_dist} = 400$ m, the matcher’s default upper bound on emission distance), compute the standard deviation of the per-fix residuals over the surviving fixes, and pick the smallest σ at which the declared emission std is no longer smaller than the measured one.

Table 1: Self-consistency check for the GPS noise scale σ on the SLC corpus.

σ_{declared}	σ_{std} (filtered)	gap = $\sigma_{\text{std}} - \sigma$	verdict
1.00	2.20	+1.20	too tight
1.75	2.77	+1.02	too tight
3.00	3.15	+0.15	slightly too tight
3.50	3.22	-0.28	first $\sigma > \sigma_{\text{std}}$ (chosen)
3.75	3.25	-0.50	too loose
4.00	3.26	-0.74	too loose
5.00	3.31	-1.69	too loose

The sign of $\sigma_{\text{std}} - \sigma_{\text{declared}}$ flips between $\sigma = 3.0$ and $\sigma = 3.5$. We therefore set $\sigma = 3.5$ m throughout this paper.

Two map versions. The pipeline operates on two views of the same SLC road network. The *low-resolution* map is the MATSim simulator network ($N \approx 99,716$ directed links). The *high-resolution* map is the same OSM extract with long or curved MATSim links subdivided into shorter sub-edges so that arterial geometry is captured by collinear sub-edges. The main pipeline uses high-resolution matching aggregated to low-resolution links; the low-resolution baseline uses low-resolution matching instead.

3.3 Population matrix

The final stage of the pipeline aggregates the matched edge visits into a sparse matrix $C \in \mathbb{Z}_{\geq 0}^{T \times N}$ that is the input to the rest of the model. Each row corresponds to a time bin, each column to a road link, and each entry counts the number of edge visits that fell into that bin and link.

Time is partitioned into non-overlapping bins of equal width $\Delta = 300$ seconds (five minutes). For an absolute timestamp t measured in seconds since the epoch $t_0 = 2018-01-01$, the bin index is $b(t) = \lfloor \frac{t-t_0}{\Delta} \rfloor$. There are $T = 30 \times 24 \times 12 = 8,640$ five-minute bins. A five-minute resolution is chosen to balance two competing constraints: too coarse a bin (e.g., one hour) blurs the rush-hour profile of the network; too fine a bin (e.g., one minute) leaves the matrix excessively sparse, with most cells empty even on busy links.

Deduplication. The map-matched output of Section 3.2 provides, for each trip r , a list of linkID (lid) - time pairs (lid_j, t_j) , one per emitting GPS location. The counting rule is:

- (1) **Bin assignment.** Each pair is assigned to the bin

$$b_j = b(t_j) = \left\lfloor \frac{t_j - t_0}{\Delta} \right\rfloor.$$

- (2) **Per-trip deduplication.** For a given trip r , all fixes that fall in the same bin and on the same link are collapsed into a single contribution. Concretely, the set of contributions from trip r is the set of distinct triples

$$\{ (b_j, \text{lid}_j, r) : j = 1, \dots, M_r \}.$$

If the same trip emits five GPS points that all map to the same bin and the same link, that combination is counted exactly once.

- (3) **Accumulation.** The population-matrix entry $C_{b,i}$ is then the number of distinct trips that contributed at least one fix to bin b on link i :

$$C_{b,i} = |\{ r : \exists j \text{ with } b_j = b, \text{lid}_j = i \}|. \quad (1)$$

The per-trip deduplication ensures that $C_{b,i}$ measures link occupancy by distinct vehicles in the corpus, not the GPS-emission density of those vehicles: a trip that produces several GPS points while crossing one link inside a single 5-minute window contributes one unit, not several.

Sparse matrix representation. The matrix C has dimensions $T \times N \approx 8.6 \times 10^8$ cells, where $T = 8,640$ is the number of five-minute bins (Section 3.3) and $N = 99,716$ is the number of directed road links. Only a small fraction of these cells are nonzero: at five-minute resolution most road links carry no trip in any specific window. We therefore store C in compressed sparse row format alongside the per-row timestamps and per-column link identifiers, which on the SLC corpus reduces the artifact to roughly 7 MiB.

Limitations of C from Bias and Sparsity. The matrix C summarizes how many trips, drawn from the 714,594-trip corpus, traversed each road link in each five-minute window of the 30-day observation period. It is a direct empirical proxy for link usage and will inform our model.

Two limitations are worth stating up front. First, the corpus is a biased sample of real traffic: it is drawn from a specific population of GPS-emitting devices and is not a uniform sample of all vehicles on the road. Links that systematically attract devices in the sample (e.g. commercial freeway corridors) are over-represented relative to links that attract a different population (e.g. residential cul-de-sacs). The matrix should therefore be read as a *prior* on link usage, not as ground-truth volume. Second, even within the large corpus, sparsity is structural: at any single five-minute slot, the median number of GPS fixes per link is zero, and only a few hundred to a few thousand links carry any matched visit at all. To quantify this at the bin-of-week level, we aggregate the first four full weeks of the corpus (8,064 bins, i.e. $4 \times 7 \times 288$ five-minute slots) and ask, for each (*link, bin-of-week*) cell, in how many of the four weeks it received at least one matched fix. The result is in Table 2.

Table 2: How often each (link, bin-of-week) cell was zero across the four weeks. Active means $k \neq 4$.

weeks with zero count	cell count	fraction
$k = 0$ (active in every week)	304,983	0.15%
$k = 1$	499,625	0.25%
$k = 2$	1,153,583	0.57%
$k = 3$	3,994,889	1.99%
$k = 4$ (never observed in 4 weeks)	195,074,376	97.04%
total	201,027,456	100%

97% of all (link, bin-of-week) cells are zero in every one of the four weeks; only 0.15% are active in all four. Most “active” cells are active in only one of the four weeks, i.e., represent a single noisy sample rather than a stable rate.

4 BUILDING THE TRAFFIC POPULATION PRIOR

The matrix C is converted into a per-bin traffic population prior by row-normalization. For every bin b with $\sum_i C_{b,i} > 0$, define

$$F_b(i) = \frac{C_{b,i}}{\sum_{i'} C_{b,i'}}.$$

This makes F_b a probability distribution over the N links of the network: the share of trip-presence mass during bin b that landed on link i . Time bins with no observed trips are assigned the uniform distribution $F_b(i) = \frac{1}{N}$, which acts as an uninformative fallback.

Sparsity and Noise Floor. The traffic population prior F_b summarizes what was actually observed on the road network during time bin b , but it has two structural shortcomings for predictive use. First, F_b is sparse: a typical 5-minute window puts non-zero mass on only a few thousand of the N links, leaving the rest of the network at exact zero. Second, the empirical signal is noisy at fine resolution: the same heavy corridor can be touched by zero observed trips in one time bin and several trips in the very next, simply because no GPS device happened to emit a fix on that link inside that 5-minute slot.

We measure the scale of this noise from the four-week structure of the corpus. For each (link, bin-of-week) cell that the corpus saw at least once, we compute the standard deviation $\tau_{b,i}$ of its normalized count $F_b(i)$ across the four weeks. This captures how much the same link in the same time-of-week jitters from week to week.

The median of $\tau_{b,i}$ over the 5.95×10^6 active cells in Table 2 is $\tau_{\text{global}} = 2.65 \times 10^{-4}$, a single corpus-wide scalar *noise floor*, used as a global yardstick of how much variation to trust. See Figure 2, which shows a log-log plot of count of these $\tau_{b,i}$ values. Differences in F_b much smaller than τ_{global} are not clearly distinguishable from week-to-week sampling noise. We use this noise floor to calibrate the smoothing step below.

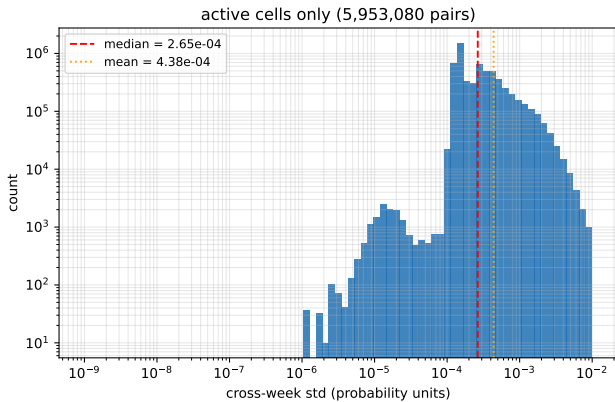


Figure 2: Distribution of the cross-week standard deviation $\tau_{b,i} = \text{std}_w E_b^{(w)}(i)$ across the four weeks, per (link, bin-of-week), restricted to the 5.95×10^6 active cells.

4.1 Diffusion Smoothing of Population Prior

A graph-aware smoothing step addresses both issues at once. It lifts every link to a positive value (so the prior never has hard zeros) and borrows mass from neighboring links (so a heavy link does not disappear because of an empty bin). The smoothing is applied row by row to the count matrix $C \in \mathbb{Z}_{\geq 0}^{T \times N}$ in three steps.

Laplace pseudo-count. For every time bin b and every link i , add a small factor $\alpha = \frac{1}{100}$ to the raw count

$$c_b(i) = C_{b,i} + \alpha, \quad \alpha = \frac{1}{100}. \quad (2)$$

After this step every entry of c_b is positive: the row is dense before any further processing.

Single-step graph diffusion. Let $P \in \mathbb{R}^{N \times N}$ be the row-stochastic adjacency matrix of the directed road graph: $P_{i,j} = \frac{1}{d_i^{\text{out}}}$ when there is a road link from i to j , and zero otherwise; here d_i^{out} is the out-degree of link i . Apply one random-walk step on the graph,

$$\tilde{c}_b = (1 - \gamma) c_b + \gamma P c_b, \quad \gamma = 0.20, \quad (3)$$

where $(P c_b)_i = \sum_j P_{i,j} c_b(j)$ is the average of $c_b(j)$ over the out-neighbors j of link i . The mass on link i is therefore averaged with the mass on the links it leads to. Finally, normalize each row to a probability distribution:

$$\tilde{F}_b(i) = \frac{\tilde{c}_b(i)}{\sum_j \tilde{c}_b(j)}. \quad (4)$$

The result $\tilde{F}_b \in \mathbb{R}^N$ is the *diffused prior*: dense, positive on every road link, and preserving the order of the heaviest links across the corpus. $\tilde{F}_b$ is used as an alternative to the raw F_b throughout the evaluation of Section 8.1.

4.2 Choosing γ from the Noise Floor

We pick the diffusion strength γ so that the smoothing rarely displaces any single cell by no more than about half of that cell's own week-to-week sampling noise. Concretely, for each (link i , bin-of-week b) cell with at least one observed positive count across the four full corpus weeks $w = 1, \dots, W$ (with $W = 4$), define the per-week displacement

$$D_{b,i}^{(w)}(\gamma) = \left| \tilde{F}_{b,i}^{(w)}(\gamma) - F_{b,i}^{(w)} \right|,$$

as the difference between the smoothed normalized count $\tilde{F}_{b,i}^{(w)}(\gamma)$ with parameter γ , and the observed count in that week $F_{b,i}^{(w)}$. The cell-level displacement averaged across the four weeks

$$D_{b,i}(\gamma) = \frac{1}{W} \sum_{w=1}^W D_{b,i}^{(w)}(\gamma),$$

and compare it to the cross-week noise floor $\tau_{b,i}$ of the same cell or the global noise floor τ_{global} . In particular, we aim for

$$\text{per-cell: } D_{b,i}(\gamma) \leq \frac{1}{2} \tau_{b,i},$$

$$\text{global: } D_{b,i}(\gamma) \leq \frac{1}{2} \tau_{\text{global}} = 1.325 \times 10^{-4}.$$

Table 3 shows statistics on this for various choices of γ . It plots the median ratio $D_{b,i}(\gamma)/\tau_{b,i}$ or $D_{b,i}(\gamma)/\tau_{\text{global}}$ (as med.). It also provides the 95th percentile of that ratio, and the percent of cells

which exceed the 0.5 criteria. The per cell criteria fairs better than the global one, indicating that error decreases with these population rates. We choose $\gamma = 0.2$ since it provides sufficient smoothing, but violates the criteria a small fraction of the time.

Table 3: Quantiles of the dimensionless displacement-to-noise ratio $D_{b,i}(\gamma)/\tau$ across the 5,954,929 active SLC cells, and percentage that exceed 0.5 ratio criteria. Left block uses the per-cell denominator $\tau_{b,i}$, right block uses the global denominator $\tau_{\text{global}} = 2.65 \times 10^{-4}$.

γ	per-cell criterion			global criterion		
	med.	95th pct.	> 0.5	med.	95th pct.	> 0.5
0.00	0.000	0.000	0.00%	0.000	0.000	0.00%
0.05	0.038	0.100	0.42%	0.034	0.215	0.90%
0.10	0.069	0.186	1.00%	0.062	0.393	3.31%
0.12	0.081	0.219	1.27%	0.073	0.463	4.40%
0.13	0.087	0.236	1.41%	0.079	0.498	4.96%
0.14	0.093	0.253	1.56%	0.084	0.533	5.53%
0.15	0.099	0.270	1.73%	0.090	0.568	6.10%
0.20	0.130	0.354	2.62%	0.117	0.741	8.95%
0.25	0.160	0.436	3.63%	0.144	0.912	11.87%
0.30	0.190	0.518	5.39%	0.171	1.081	14.81%
0.40	0.248	0.680	9.42%	0.224	1.413	20.64%
0.50	0.306	0.838	17.29%	0.276	1.737	26.59%
0.70	0.418	1.145	32.10%	0.376	2.364	37.83%

JEFF: We probably don't need this zero cell data

Table 4: [temporary] Displacement of zero cells ($n_{\text{zero}} = 1.95 \times 10^8$) in units of $\tau_{\text{global}} = 2.65 \times 10^{-4}$.

γ	median	95th pct.	> 0.5
0.00	0.0004	0.0004	0.00%
0.05	0.0004	0.0004	0.01%
0.10	0.0004	0.0004	0.03%
0.12	0.0004	0.0004	0.04%
0.13	0.0004	0.0004	0.04%
0.14	0.0004	0.0004	0.05%
0.15	0.0004	0.0004	0.05%
0.20	0.0004	0.0004	0.08%
0.25	0.0004	0.0004	0.10%
0.30	0.0004	0.0004	0.13%
0.40	0.0004	0.0004	0.20%
0.50	0.0004	0.0004	0.28%
0.70	0.0003	0.0004	0.46%

Reading the zero-cell table. The median and 95th-percentile ratios are essentially constant at $\approx 4 \times 10^{-4}$ across all γ : at least 95% of zero cells never see diffusion mass, they only carry the Laplace floor α/N . The fraction of zero cells exceeding the half-noise tolerance is 0.08% at $\gamma = 0.20$ ($\sim 1.5 \times 10^5$ cells out of 1.95×10^8); these are the cells immediately adjacent to active links and they are the ones that get visibly filled in. The takeaway is that the diffusion is bounded even on the cells where the per-cell criterion of Table 3 is undefined: it lifts a small adjacency-defined tail of zero cells into the noise band and leaves the rest at the Laplace floor.

Violator tail at $\gamma = 0.20$. To see how far past the tolerance the violators sit, we sweep the threshold η in $D_{b,i}(0.20) > \eta \tau_{\text{global}}$ across the active cells:

η	0.2	0.5	0.8	1.0	1.5	2.0
fraction $> \eta \tau_{\text{global}}$	28.5%	8.9%	4.4%	3.0%	1.3%	0.6%

The violator tail thins quickly: of the 8.9% that exceed half the noise floor, only 3% exceed τ_{global} and fewer than 0.7% exceed $2 \tau_{\text{global}}$, so the smoothing rarely changes a cell by more than its own week-to-week jitter.

4.3 Evaluation Mechanism

Now given the diffused population prior $\tilde{F}_b$, which we use as our ground truth proxy $t_i = \tilde{F}_b(i)$, we set up how we measure our model. Since the population of each cell still varies significantly at different scales, we use a relative error for a prediction p_i at a time bin i is,

$$r_i = \frac{|t_i - p_i|}{t_i + \varepsilon},$$

where $\varepsilon = 10^{-6}$ is included to add stability. Note this less than $\tau_{\text{global}}/100$, less than the effect of $\frac{1}{10}$ th of a trip on the busiest cell. Then the overall score at that time bin b is the mean relative error

$$\text{MRE} = \frac{1}{N} \sum_{i=1}^N r_i.$$

One week to next. We frame our predictions by using data from one week to predict the next. This is similar to the noise floor, where we calibrated against the raw data set statistics predicting the next week at the same link and same time/day. For our prediction, we use a forecast for a model built on one week, predicting to the same time period exactly 7 days forward (move forward time index by 2016). For global analysis, we train parameters on the first 23 days of the dataset, aggregate them to the days/times of a single week, and the predict on the final seven days of the week, averaged across all 2,016 of the 5 minute time windows.

5 PAGERANK MODEL FOR LINK-LEVEL TRAFFIC

We now describe our PageRank model for representing traffic patterns on our metro-scale network. Recall that PageRank [] was developed as a model of a random surfer on the internet; the resulting distribution of where the random surfer would visit in the limit formed the basis for Google's groundbreaking search engine. In this way it models the outgoing links of a webpage as directed edges in a graph, with pages as N nodes; these form the probability transition matrix $P \in \mathbb{R}^{N \times N}$. The i th column $P_i \in \mathbb{R}^N$ is a probability distribution for node i where $P_i[j]$ encodes the probability that a random walk at node i will be in node j in the next step. In basic PageRank, these probabilities are set uniformly among the outgoing hyperlinks of a webpage, so P is very sparse.

Just using P does not define an ergodic Markov Chain, so a key innovation is having the random surfer *teleport* to a random webpage with some probability p (usually about 0.15). This makes the random walks follow a geometric distribution in length, with expected length $1/p$ (about 7 steps on average). It also makes the true

transition probability dense, but still allowing efficient simulation of forward steps from initiate state v_n as:

$$v_{n+1} = pE + (1 - p)Pv_n. \quad (5)$$

Here $E \in \mathbb{R}^{N \times N}$ represents the uniform random distribution, so it is the $1/N$ times the all 1 matrix. However, note that this step can be implemented by replacing pE with a fixed vector $e \in \mathbb{R}^N$ that has p/N in each coordinate.

5.1 Baseline Traffic PageRank Model

Our baseline traffic model is to adopt this PageRank model to our large roughly 100,000 ($N = 99,716$) link network. Since our population data mapped to the road network links, these links map to the *nodes* in the graph, not the edges. The intersections (and direct connection between consecutive links) are the edges. This switch (links $\neq$ edges) is slightly un-intuitive, but allows for the precise sort of modeling we seek, and aligns with our population date mapping to links.

Then we can define the probability transition matrix $P \in \mathbb{R}^{N \times N}$ by the road network; we make the probability transition uniform $1/k$ among all (k) options from a directed link. Then with $E \in \mathbb{R}^{N \times N}$ with each entry $E_{i,j} = 1/N$ our transition update follows (5) precisely.

In addition to this modeling being able generate realistic routes, it can also generate global traffic distributions. By iteratively applying (5), the $\lim_{n \rightarrow \infty} v_n v_\star$ provides the modeled distribution of traffic across all road network links.

Calibrating p from the trip corpus. We next need to calibrate p from the trip corpus. The chain in (5) is a geometric absorption process: the number of graph steps before a teleport is $\text{Geom}(p)$ on $\{0, 1, 2, \dots\}$ with mean $\frac{1-p}{p}$. We match p to the empirical mean trip length $\mathbb{E}[K_{\text{total}}]$ by method-of-moments:

$$p = \frac{1}{1 + \mathbb{E}[K_{\text{total}}]}.$$

On the map-matched corpus (714,594 trips, all route lengths), the empirical mean is $\mathbb{E}[K_{\text{total}}] = 18.68$, which gives $p = 0.0508$. We thus use this for this $p = 0.0508$ going forward. Figure 3 shows the empirical K_{total} histogram with the $\text{Geom}(p = 0.0508)$ reference overlaid, illustrating the nice geometric fit to true route length distribution.

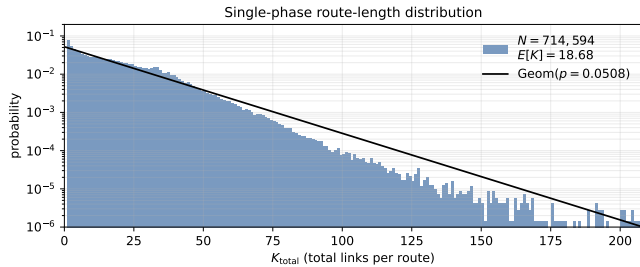


Figure 3: Empirical histogram of trip length K_{total} on the 714,594-trip corpus, with the $\text{Geom}(p = 0.0508)$ overlay.

Implementation Details. Given the sparse probability transition probability matrix P of the directed road graph (each column of P being uniform over the outgoing road neighbors of one link) and a teleport probability $p = 0.0508$, the iteration in (5) is applied until $\|v_{n+1} - v_n\|_1 < 10^{-7}$, with a safety cap of $n = 300$ iterations. The cap is conservative: the power iteration converges geometrically at rate of at least $1 - p$, so the theoretical worst-case number of iterations to reach L1 error 10^{-7} is

$$n^\star = \frac{\ln 10^{-7}}{\ln(1 - p)} = \frac{\ln 10^{-7}}{\ln 0.9492} \approx 310.$$

We checked empirically on 1,500+ random target bins and the chain converged before 300 iterations in every single case; the observed maximum was 254 iterations and the median was 209.

5.2 PageRank Model with Informed Teleportation

Beyond this baseline model, the first modification we make is that the teleportation step is not uniform. Which links to start routes are not uniform. Its quite rare to *start* a route on a busy freeway, and it should be more likely to start one from a grocery store parking lot.

For each time window b we can track all routes that start in that period. After normalizing these counts we can get a distribution $e_b \in \mathbb{R}^N$ to get a probability distribution. Like the traffic population matrix F_b , this also faces an extreme sparsity challenge. So we apply the same graph diffusion method (as in Section 4.1) to get a diffuse route initialization distribution $\tilde{e}_b$.

Our first innovation replaces the uniform random teleportation matrix with the population diffusion model of routes $\tilde{E}_b$, where each column is $\tilde{e}_b$. Then the simulation update step is

$$v_{n+1} = p\tilde{E}_b + (1 - p)Pv_n. \quad (6)$$

This accomplishes two important aspects. First, it adapts the simulation to start routes at more realistic locations informed by actual data. Second, allows the model to adapt to the specific time-window b ; so the traffic model can generate different routes at different times during the day or week.

5.3 PageRank Model with Learned Transitions

We next apply machine learning to learn a more informed probability transition matrix P . A potentially unrealistic part of a PageRank model is that the routes are random walks, while realistic routes are directed and meaningful. However, the Markov chain modeling requires the *Markov* properties that each step is memoryless, so we cannot use this entire set up if we inform an origin-destination pair. However, we can adapt the transition probabilities to reflect more realistic to choices that model traffic. We do so with two modeling aspects following the principals that routes tend to move towards higher-throughput links.

These aspects leverage physical throughput attributes of links ℓ of the graph: flow speed $s(\ell)$ and a lane count $\lambda(\ell)$. Let $\bar{s}(\ell) = s(\ell)/\bar{s}$ and $\bar{\lambda}(\ell) = \lambda(\ell)/\bar{\lambda}$ denote these attributes normalized by their per-link mean over the network.

Now given from a link ℓ_i the weight of each potential outgoing link ℓ_j is parameterized by 3 parameters α_s, α_l so that

$$w(j) = \bar{s}(\ell_j)^{\alpha_s} \bar{\lambda}(\ell_j)^{\alpha_l}.$$

Finally, we can set the transition probability matrix from i to j among all outgoing edges J_i from i as

$$P_{j,i} = \frac{w(j)}{\sum_{j \in J_i} w(j)}.$$

And P retains its sparsity with $P_{j,i} = 0$ for $j \notin J_i$.

It remains to set the values for $\alpha = (\alpha_s, \alpha_l)$. These values α_s, α_l are larger when there is a preference to turn onto a link with higher throughput (because of higher flow-rate e.g., speed limit, or number of lanes). A negative value in α_s, α_l would indicate a preference to turn onto smaller roads.

Towards determining these, we set up a loss function

$$\mathcal{L}(\alpha) = \sum_{b \in [T]} \sum_{i \in [N]} \frac{|v_\star(\tilde{E}_b, \alpha)[i] - \tilde{F}_b[i]|}{\tilde{F}_b[i] + \varepsilon}$$

where $v_\star(\tilde{E}_b, \alpha)$ is the converged-to PageRank vector for a Markov chain following (6), where P is defined using the specified α parameters. Given the diffused starting positions $\tilde{E}_b$, the goal is to match the diffused observed traffic population $\tilde{F}_b$. It is the MRE (mean relative error), summed over all time periods b ; thus the goal is to set these α parameters globally for all timescales; that is to learn the driving preferences for the metro area regardless of the time of day. While it may be the case that these driving patterns change throughout the day or week, we opt to learn a smaller set of parameters with goal of overfitting. Moreover, this training step is computationally expensive, and repeating it 8000+ times, would be exorbitant.

Finally, we can choose these parameters $\alpha^* = \arg \min_{\alpha} \mathcal{L}(\alpha)$. We do this with gradient descent. We use autograd to determine the gradient, and try several random restarts to confirm it reliably finds the same minimal parameter choices.

5.4 Two-phase PageRank Model

The final aspect of our model is based on the observation that most traffic routes gradually move to higher throughput links from the origin, but the destination is usually not on these high throughput links. As it approaches a destination, the route tends to move towards lower throughput links. However, the memoryless aspect of PageRank models does not directly allow for a goal destination or these two phases.

Towards addressing this, we describe a two-phase PageRank model. Instead of a N state model, this becomes a $2N$ state model, where each link can be visited in the *up* phase as the route tends to move towards higher throughput links as it leave the origin, or in the *down* phase as the route tends to move towards lower throughput links as it approaches the destination. The up phase is now governed by a probability transition matrix $P^\uparrow$ with parameters $\alpha^\uparrow = (\alpha_s^\uparrow, \alpha_l^\uparrow)$, and the down phase by a separate $P^\downarrow$ with its own parameters $\alpha^\downarrow = (\alpha_s^\downarrow, \alpha_l^\downarrow)$. Then instead of a single parameter p to govern the probability the route ends, we need two parameters: β and ρ . The β parameter is the probability that when in the up phase the route transitions to the down phase, and ρ parameter is the probability that when in the down phase the route reaches its destination and ends. While after the down phase ends, the route needs to teleport to a new starting spot (based on $\tilde{E}_b$), when the up phase ends the route should stay on the same link as before. We can

capture this entire in a transition probability matrix $M_b \in \mathbb{R}^{2N \times 2N}$, where each component is an $N \times N$ block:

$$M_b = \begin{pmatrix} (1 - \beta) P^\uparrow & \rho \tilde{E}_b \mathbf{1}^\top \\ \beta I & (1 - \rho) P^\downarrow \end{pmatrix}, \quad (7)$$

where I is the $N \times N$ identity and $\mathbf{1} \in \mathbb{R}^N$ is the all-ones column vector. With this convention the Markov chain's mass distribution $v \in \mathbb{R}^{2N}$ evolves as the simple matrix-vector product (which can be still implemented block-wise with efficient sparse operations):

$$v_{n+1} = M_b v_n. \quad (8)$$

Note that since state distribution v_n and the converged to $v_\star$ is now $2N$ -dimensional, in order to compare to a desired diffused population state $\tilde{F}_b \in \mathbb{R}^N$, we need to "fold" $v_\star$ into $\mathbb{R}^N$. That is let $\hat{v}_\star \in \mathbb{R}^N$ be defined for index $i \in [N]$ as $\hat{v}_\star[i] = v_\star[i] + v_\star[N + i]$. That is we add together the probabilities of being in the up state $i^\uparrow = i$ and the down state $i^\downarrow = N + i$.

Calibrating β and ρ . The two scalars β and ρ control how long the chain stays in each phase before switching. However, the sum of two geometric distributions follows a negative binomial distribution which does not match the observations in Figure 3; the negative binomial will have peak probability away from 0. We would rather like to interpret their ideal values as $\beta = \frac{1}{1 + \mathbb{E}[L_\uparrow]}$ and $\rho = \frac{1}{1 + \mathbb{E}[L_\downarrow]}$, where $L_\uparrow$ and $L_\downarrow$ are the empirical up- and down-phase path lengths. To do so, we need to model the lengths the up and down phase separately from the corpus.

Every matched route is a sequence of road links $(\ell_1, \ell_2, \dots, \ell_K)$ produced by the Viterbi pass of Section 3.2. For each trip we identify a single *peak link* ℓ_p as the link with the largest score $s(\ell) = \text{freespeed}(\ell) \times \text{numLanes}(\ell)$, with ties broken by the link closest to the trip's middle. The peak link plays the role of the trip's fastest, highest-capacity segment: the freeway core of a typical commute. The two phases of the chain are then defined as the prefix and the suffix of the route relative to this peak,

$$L_\uparrow = |\{j : j < p\}|, \quad L_\downarrow = |\{j : j > p\}|, \quad (9)$$

so $L_\uparrow + 1 + L_\downarrow = K$. By construction $L_\uparrow, L_\downarrow \in \{0, 1, 2, \dots\}$: a route can begin or end on its own peak link, in which case the corresponding phase has length zero. All 714,594 matched routes are therefore included in the calibration sample.

Figure 4 shows the histograms of $L_\uparrow$ and $L_\downarrow$ on this sample, with the fit geometric reference curve overlaid in black on each panel. Both empirical distributions are heavy at zero and decay roughly exponentially, which is the signature of a geometric distribution. The black curves overlay $\text{Geom}(p)$ with $p = \frac{1}{1 + \mathbb{E}[L]}$ using the empirical mean $\mathbb{E}[L]$. The empirical means are $\mathbb{E}[L_\uparrow] = 8.81$ and $\mathbb{E}[L_\downarrow] = 8.87$ for the up and down phases, respectively lead to $\beta = \frac{1}{1 + \mathbb{E}[L_\uparrow]} = 0.102$ and $\rho = \frac{1}{1 + \mathbb{E}[L_\downarrow]} = 0.101$. These estimated values $\beta, \rho \approx 0.10$, correspond to chains that stay in each phase about 10 steps on average.

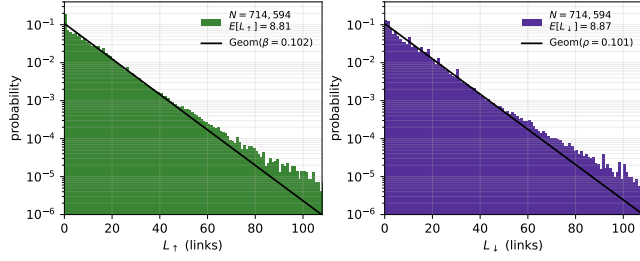


Figure 4: Empirical histograms of $L_{\uparrow}$ (left) and $L_{\downarrow}$ (right) on the full 714,594-trip matched corpus, with the matched geometric overlay in black.

Quantifying the fit. We pair the visual fit of Fig. 4 with the Kolmogorov-Smirnov statistic $D = \sup_k |F_{\text{emp}}(k) - F_{\text{geom}}(k; p)|$, where $F_{\text{emp}}(k)$ and $F_{\text{geom}}(k; p) = 1 - (1 - p)^{k+1}$ is the CDF of the fitted geometric. Smaller D means a tighter empirical match. Table 5 reports D in three configurations. Column D_{single} fits one geometric to the full route length $K = L_{\uparrow} + 1 + L_{\downarrow}$; columns $D_{\uparrow}$ and $D_{\downarrow}$ fit geometrics to $L_{\uparrow}$ and $L_{\downarrow}$ separately. The first row matches on the low-resolution map and aggregates to low-resolution links; the second matches on the high-resolution map and aggregates to the same links; the third both matches and aggregates at high-resolution. The two-phase model with the high-res matcher on the low-res links (that we use) has the smallest score, and is the best fit.

Table 5: Kolmogorov-Smirnov distance D between the single or two-phase histograms and the fitted geometric model.

Matcher	Granularity	D_{single}	$D_{\uparrow}$	$D_{\downarrow}$
low-res	low-res links	0.188	0.129	0.103
high-res	low-res links	0.129	0.030	0.032
high-res	high-res sub-edges	0.102	0.138	0.148

Fitting Two-Phase PageRank Model. Finally, to complete the two-phase PageRank model, we need to fit the parameters $\alpha^{\uparrow}$ and $\alpha^{\downarrow}$ for the full model to be complete. As in the single phase model, we set up a loss function

$$\mathcal{L}(\alpha) = \sum_{b \in [T]} \sum_{i \in [N]} \frac{|\hat{v}_{\star}(\tilde{E}_b, \alpha)[i] - \tilde{F}_b[i]|}{\tilde{F}_b[i] + \varepsilon}$$

where $\hat{v}_{\star}(\tilde{E}_b, \alpha) \in \mathbb{R}^N$ now the folded, converged-to two-phase PageRank vector for a Markov chain following (8), where $P^{\uparrow}$ and $P^{\downarrow}$ are defined with the specified $\alpha^{\uparrow}$ and $\alpha^{\downarrow}$ parameters where now $\alpha = (\alpha^{\uparrow}, \alpha^{\downarrow}) = (\alpha_s^{\uparrow}, \alpha_l^{\uparrow}, \alpha_s^{\downarrow}, \alpha_l^{\downarrow}) \in \mathbb{R}^4$. Given the diffused starting positions $\tilde{E}_b$, the goal is still to match the diffused observed traffic population $\tilde{F}_b$. As before we choose these parameters α^* = $\arg \min_{\alpha \in \mathbb{R}^4} \mathcal{L}(\alpha)$. We do this with gradient descent via autograd.

6 GLOBAL FORECASTING EVALUATION

We now evaluate our model on how well it predicts the global traffic distribution over the course of a week. We use MRE as described

in section 4.3. More specifically, we will evaluate the model using the last 7 days of data. We compute the average MRE value of the model $\hat{v}_{\star}(\tilde{E}_{b-7d}, \alpha)$ against the diffused data $\tilde{F}_b$. Note that the index of $\tilde{E}_{b-7d}$ indicates it is the diffused observation of starting routes from precisely 7 days earlier in time than the ones evaluated $\tilde{F}_b$. However, the parameters α are learned using, averaged over the first 23 days of observations. This maintains a train-test split.

More specifically we compute the average relative error for each link $i \in [N]$ for every b in the last week as

$$\text{MRE}_b = \frac{1}{N} \sum_{i=1}^N \frac{|\tilde{F}_{b,i} - v_{\star}(\tilde{E}_{b-7d}, \alpha)|}{\tilde{F}_{b,i} + \varepsilon}.$$

The Overall MRE (Ov.MRE) is reported by consider 5 minute time bins $b \in [T_{-7d}]$ for $T_{-7d} = [2018-09-23 \text{ 19:00}, 2018-09-30 \text{ 18:55}]$ which are in the last week over observations, held out for evaluation. That is we compute and report

$$\text{Ov.MRE} = \frac{1}{T_{-7d}} \sum_{b \in [T_{-7d}]} \text{MRE}_b.$$

Table 6 shows the Overall MRE under various settings and baselines. The top half shows the Single Phase PageRank model, and the bottom half shows the Two Phase PageRank Model. The uniform row uses the uninformed teleportation process, where it starts a route at a uniform random link, each with probability $1/N$; it has no other tuning. The origin row replaces this with the diffused route origin prior from 7 days prior $\tilde{E}_{b-7d}$. This allows the model to adapt to the specific 5 minute time window b with the week. The remain rows also use this prior $\tilde{E}_{b-7d}$, and also run various α parameters. While the last row speed + lane + angle runs all three α parameters, the other rows only tune a subset of them, with the others set to $\alpha_i = 0$ so that modeling as no effect on P .

The Table shows the train and test Overall MRE for each setting. Note that even the uniform (which uses no information from the data other than the graph structure) has higher Overall MRE on test than train. So while it is not surprising for test scores to be higher than training scores (due to perhaps slight overfitting on the training set) we think most of this difference is due to the inherent nature of the evaluate split we chose.

We observe that each of the enhancements over the baseline (uniform) model provide some benefit in the evaluation. **JEFF: TBD ... need to write once numbers are all in and stable.**

7 EVENT MODELING

We show how to apply this framework to model to specific events that have predictable configuration changes, which allows for updated modeling.

There are two direct ways we can leverage our model to provide what-if planning for different sorts of scenarios. First, if there are road closures (either planned or unplanned), then we can perform a simple modification of the graph by removing those links. Second, if there is a large event attracting many of additional people to a small area, then we can adapt the route origin distribution $\tilde{E}_b$ for the specified time windows b for the event. In practice for sporting events or festivals we can use prior events for these modified distribution estimates, but because we only have access to one month of data, we use the data from the $\tilde{E}_b$ data actual event. In both

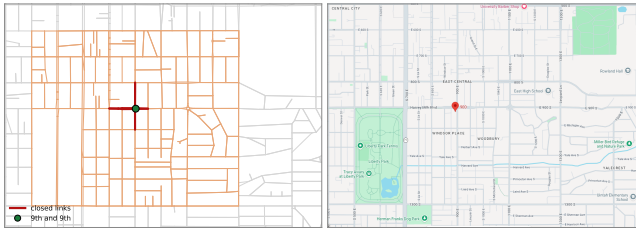
Table 6: Origin-informed within-week forecast: trained on the first 23 days, evaluated on the last 7.

mode	α_s	α_l	α_{ang}	train mean	train median	test mean	test median
<i>Single Phase</i>							
uniform: $E = 1/N$	—	—	—	1.1739	0.9544	1.2706	1.0545
origin: $\tilde{E}_{b-7d}$	—	—	—	1.1266	0.9311	1.2178	1.0277
speed	+2.242	—	—	1.1096	0.9179	1.1963	1.0109
lane	—	+0.673	—	1.1136	0.9209	1.2018	1.0153
speed + lane	+1.751	+0.436	—	1.1045	0.9138	1.1903	1.0057
lane + angle	—	+1.790	-2.507	1.1092	0.9496	1.1899	1.0383
speed + angle	+4.890	—	-2.198	1.1058	0.9454	1.1861	1.0345
speed + lane + angle	+1.860	+0.438	+0.105	1.1042	0.9132	1.1897	1.0048
<i>Two Phase</i>							
uniform: $E = 1/N$	—	—	—	1.1732	0.9540	1.2698	1.0540
origin: $\tilde{E}_{b-7d}$	—	—	—	1.1211	0.9278	1.2117	1.0243
speed	+2.201	—	—	1.1049	0.9162	1.1915	1.0084
lane	—	+0.659	—	1.1091	0.9187	1.1969	1.0125
speed + lane	+1.720	+0.416	—	1.1006	0.9117	1.1864	1.0039
lane + angle	—	+1.654	-2.216	1.1026	0.9427	1.1831	1.0315
speed + angle	+4.579	—	-1.949	1.0979	0.9376	1.1781	1.0259
speed + lane + angle	+1.781	+0.417	+0.057	1.1006	TBD	1.1862	TBD

cases, we do not need to retrain the α parameters, only to rerun the PageRank iteration (8); which is fast because we maintain a sparse format, and the distribution vector $v_\star$ is converged to quickly.

7.1 Case study 1: Street Festival

The 9th and 9th Street Festival is an annual neighborhood event in Salt Lake City centered on 900 South and 900 East. For its 2018 edition (Saturday September 15) the organizers closed 900 East from 800 South to Belmont and 900 South from 800 East to Lincoln to through traffic.¹ ?? shows the 14-link cross at the festival hub on the SLC road graph and on a Google Maps view of the neighborhood. We treat the closure as a graph-surgery intervention with the chain’s trained exponents frozen: on every 5-minute slot of that Saturday the model is run once on the original road graph and once on a surgery graph in which the closed links are deleted, the entries of the diffused prior E_b and of the diffused popularity F_b on those links are zeroed, and all three vectors are re-normalised over the open links.

**Figure 5: Caption**

¹Closure boundaries reported by the festival organizers, <https://9thand9thstreetfestival.com/accessibility/>.

Table 7: Festival surgery. Closed links are removed from P , from the diffused prior E_b (re-normalised over open), and from the diffused popularity F_b (re-normalised over open). The chain output is re-normalised the same way before scoring. 2018-09-15, 10:00 to 17:55, 96 slots.

region	links	ORIG mean	ORIG median	SURG mean	SURG median	Δ mean
whole network	99,702	1.3263	1.3448	1.3263	1.3448	0.0000
eval box	694	1.4739	1.4961	1.4872	1.5074	+0.0133

Table 8: Festival prior-zero: P unchanged, the diffused prior E_b is zeroed on the closed links and re-normalised, the diffused popularity F_b is left untouched. 2018-09-15, 10:00 to 17:55, 96 slots.

region	links	ORIG mean	ORIG median	ZERO mean	ZERO median	Δ mean
whole network	99,716	1.3262	1.3446	1.3262	1.3446	0.0000
eval box	708	1.4860	1.5039	1.4519	1.4700	-0.0341

8 ABLATION / APPENDIX

The evaluation framework (Section 4.3) defines the metric (Overall MRE), the two comparisons (dataset-to-dataset, forecast), and the 840-bin sampling protocol used by every table below.

8.1 Evaluation results

Table 9 reports the three comparisons in three configurations: raw prior with $\varepsilon = 10^{-6}$, diffused prior with $\varepsilon = 10^{-6}$, and diffused prior with $\varepsilon = 0$. The chain uses the calibrated parameters $\beta = 0.102$, $\rho = 0.101$ and $P^\uparrow, P^\downarrow$ uniform over outgoing road-graph neighbors. Iteration tolerance is 10^{-7} , maximum 300 iterations.

Table 9: Overall MRE of the two-phase chain across 840 random target bins (120 per day of the week) on the SLC corpus. $\beta = 0.102, \rho = 0.101$. **Lower is better. “No diffusion” uses the raw prior E_b of Section 3.3; “Diffused” uses the diffused prior $\tilde{E}_b$ of Section 4.1.**

Comparison	No diffusion $\varepsilon = 10^{-6}$	Diffused $\varepsilon = 10^{-6}$	Diffused $\varepsilon = 0$
1. dataset→dataset (next week)	5.82	0.68	0.90
2. dataset→model (forecast)	8.53	1.04	1.42

Two readings of the table.

Forecast sits above the noise floor. Row 2 forecast Overall MRE is 1.04, above the noise floor of row 1 at 0.68. The chain’s forecast of next week’s signal is therefore worse than last-week-at-the-same-slot. Since the chain has no temporal dynamics (no time index in M_b except through the prior of the target bin), improving forecast performance would require additional signal: weekly periodicity, multi-bin context, or external covariates, none of which are part of the current model.

Empty-link contribution to the overall MRE. The Overall MRE without the regularizer inflates to ~ 8 in the leftmost column because the many empty links contribute $\frac{|p_i|}{\varepsilon}$ each; $\varepsilon = 10^{-6}$ caps this contribution but does not eliminate it. With every truth value positive, the Overall MRE drops to 0.67–1.37, a range in which the metric is interpretable as a relative error average.

8.2 Additive ladder: road-type tuning with calibrated chain dynamics

This subsection isolates the contributions of damping and road-type weighting on the same evaluation sample. The sample is the set of 840 calendar pairs ($t, t+7$ d) used throughout this section, drawn so that exactly 120 pairs sit on each day of the week (seed 7), with t chosen from the first 23 days of the corpus so that the next-week sibling is in range. The model receives $\tilde{E}_b(t)$ and predicts $\tilde{E}_b(t+7$ d). The per-pair error is the Overall MRE (over all N links), and we report its mean across the 840 pairs.

Damping sweep (no road-type tuning). We first hold $\alpha_s = \alpha_l = 0$ and sweep the damping rate. For the single-phase chain the rate is α . For the two-phase chain we set $\beta = \rho$ and sweep the common value, so that the two columns differ only by the chain structure.

Table 10: Damping sweep on the whole network, 840 pairs ($t, t+7$ d), no road-type weighting. Overall MRE (lower is better); bold marks the best in each column.

rate	Single-phase (α)	Two-phase ($\beta = \rho$)
0.03	1.130	1.181
0.04	1.098	1.154
0.05	1.072	1.131
0.06	1.049	1.110
0.07	1.029	1.091
0.15	0.923	0.984

Both columns improve monotonically as the damping rate increases toward the upper bound of the search box. With *no road-type weighting* single-phase outperforms two-phase at every common rate by approximately 0.05–0.06 on Overall MRE.

Road-type tuning with calibrated chain dynamics. Table 11 flips the comparison: the chain dynamics are now *pinned* at the geometric-fit values estimated from the data ($\alpha = 0.0508$ for the single-phase chain; $\beta = 0.102, \rho = 0.101$ for the two-phase chain in its main-body form). Only the road-type exponents α_s and α_l are learned by the optimizer (L-BFGS-B, five random starts, finite-difference gradients, unbounded). Five rows are shown per chain:

- *uniform prior:* $E_b = 1/N$, no road-type tuning;
- *diffuse prior:* $E_b = \tilde{E}_b(t)$, no road-type tuning;
- *+lanes only, +speed only, +both:* progressively add the road-type exponents.

The two blocks of the table differ only by the chain. Once the dynamics are calibrated from the data, the picture in Table 10 reverses: the two-phase chain beats single-phase at every row of the ladder by about 0.02–0.03 on Overall MRE, with the gap widening once road-type weights are introduced.

Table 11: Single-phase (SP) vs. two-phase new-form (TP) on the 840-pair eval, with chain dynamics fixed at the geometric-fit values. The optimizer only learns the road-type exponents α_s, α_l (unbounded); the chain parameters α, β, ρ are never touched. “–” means no road-type tuning. TP beats SP at every row.

variant	parameters at the optimum	Ov. MRE
<i>Single-phase</i> ($\alpha = 0.0508$ fixed)		
uniform prior	$E_b = 1/N$	1.3127
diffuse prior	E_b from the diffused popularity	1.0697
+ lanes only	$\alpha_l = +0.411$	1.0596
+ speed only	$\alpha_s = +3.050$	1.0028
+ both	$\alpha_s = +2.775, \alpha_l = +0.242$	0.9997
<i>Two-phase (new form)</i> ($\beta = 0.102, \rho = 0.101$ fixed)		
uniform prior	$E_b = 1/N$	1.3119
diffuse prior	E_b from the diffused popularity	1.0413
+ lanes only	$\alpha_l = +0.377$	1.0323
+ speed only	$\alpha_s = +3.052$	0.9725
+ both	$\alpha_s = +2.816, \alpha_l = +0.210$	0.9701

REFERENCES

- [1] Wannes Meert and Mathias Verbeke. HMM with non-emitting states for map matching. In *European Conference on Data Analysis (ECDA)*, Paderborn, Germany, July 2018.
- [2] Wannes Meert. *Leuven Map-Matching: a Python library for matching GPS trajectories to road networks*. KU Leuven, DTAI Research Group, 2015–2024. <https://github.com/wannesm/LeuvenMapMatching>.
- [3] Paul Newson and John Krumm. Hidden Markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '09)*, pages 336–343. ACM, 2009. doi:10.1145/1653771.1653818.

- [4] Roger W. Sinnott. Virtues of the Haversine. *Sky and Telescope*, 68:158, 1984. <https://ui.adsabs.harvard.edu/abs/1984S%26T...68..158S/abstract>.
- [5] John P. Snyder. *Map Projections: A Working Manual*. U.S. Geological Survey Professional Paper 1395, U.S. Government Printing Office, Washington, DC, 1987. Chapter 12, “Equidistant Cylindrical Projection”, pp. 90–91. <https://pubs.usgs.gov/pp/1395/report.pdf>.
- [6] Mohammed A. Qudus, Washington Y. Ochieng, and Robert B. Noland. Current map-matching algorithms for transport applications: state-of-the-art and future research directions. *Transportation Research Part C: Emerging Technologies*, 15(5):312–328, 2007. doi:10.1016/j.trc.2007.05.002.
- [7] Yin Lou, Chengyang Zhang, Yu Zheng, Xing Xie, Wei Wang, and Yan Huang. Map-matching for low-sampling-rate GPS trajectories. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '09)*, pages 352–361. ACM, 2009. doi:10.1145/1653771.1653820.
- [8] Taras Agryzkov, Leandro Tortosa, and José F. Vicent. Extending the Adapted PageRank Algorithm centrality model for urban street networks using non-local random walks. *Applied Mathematics and Computation*, 447:127531, 2023. doi:10.1016/j.amc.2023.127531.
- [9] Marta C. González, César A. Hidalgo, and Albert-László Barabási. Understanding individual human mobility patterns. *Nature*, 453:779–782, 2008. doi:10.1038/nature06958.
- [10] Dirk Helbing. Traffic and related self-driven many-particle systems. *Reviews of Modern Physics*, 73(4):1067–1141, 2001. doi:10.1103/RevModPhys.73.1067.
- [11] Md. Shahadat Iqbal, Charisma F. Choudhury, Pu Wang, and Marta C. González. Development of origin-destination matrices using mobile phone call data. *Transportation Research Part C: Emerging Technologies*, 40:63–74, 2014. doi:10.1016/j.trc.2014.01.002.
- [12] Xiangdong Yang, Yang Lu, and Wei Hao. Origin-destination estimation using probe vehicle trajectory and link counts. *Journal of Advanced Transportation*, 2017:4341532, 2017. doi:10.1155/2017/4341532.
- [13] Pu Wang, Timothy Hunter, Alexandre M. Bayen, Katja Schechtner, and Marta C. González. Understanding road usage patterns in urban areas. *Scientific Reports*, 2:1001, 2012. doi:10.1038/srep01001.
- [14] Yu Zheng. Trajectory data mining: an overview. *ACM Transactions on Intelligent Systems and Technology*, 6(3):29, 2015. doi:10.1145/2743025.
- [15] Jing Yuan, Yu Zheng, Chengyang Zhang, Wenlei Xie, Xing Xie, Guangzhong Sun, and Yan Huang. T-drive: driving directions based on taxi trajectories. In *Proceedings of the 18th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '10)*, pages 99–108. ACM, 2010. doi:10.1145/1869790.1869807.
- [16] Daqing Li, Bowen Fu, Yunpeng Wang, Guangquan Lu, Yehiel Berezin, H. Eugene Stanley, and Shlomo Havlin. Percolation transition in dynamical traffic network with evolving critical bottlenecks. *Proceedings of the National Academy of Sciences (PNAS)*, 112(3):669–672, 2015. doi:10.1073/pnas.1419185112.
- [17] Erik Jenelius, Tom Petersen, and Lars-Göran Mattsson. Importance and exposure in road network vulnerability analysis. *Transportation Research Part A: Policy and Practice*, 40(7):537–560, 2006. doi:10.1016/j.tra.2005.11.003.
- [18] Vito Latora and Massimo Marchiori. Vulnerability and protection of infrastructure networks. *Physical Review E*, 71:015103(R), 2005. doi:10.1103/PhysRevE.71.015103.
- [19] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. The emerging field of signal processing on graphs: extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013. doi:10.1109/MSP.2012.2235192.
- [20] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems (NeurIPS)* 29, pages 3837–3845, 2016. <https://arxiv.org/abs/1606.09375>.
- [21] Andreas Horni, Kai Nagel, and Kay W. Axhausen (editors). *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press, London, 2016. doi:10.5334/baw.
- [22] Michael Balmer, Kay W. Axhausen, and Kai Nagel. Agent-based demand-modeling framework for large-scale microsimulations. *Transportation Research Record: Journal of the Transportation Research Board*, 1985:125–134, 2006. doi:10.1177/0361198106198500114.
- [23] Minjie Wang, Su Yang, Yi Sun, and Jun Gao. Discovering urban mobility patterns with PageRank based traffic modeling and prediction. *Physica A: Statistical Mechanics and its Applications*, 485:23–34, 2017. doi:10.1016/j.physa.2017.04.155.
- [24] Bin Jiang. Ranking spaces for predicting human movement in an urban environment. *International Journal of Geographical Information Science*, 23(7):823–837, 2009. doi:10.1080/13658810802022822.
- [25] Armando Bazzani, Bruno Giorgini, Sandro Rambaldi, Riccardo Gallotti, and Luca Giovannini. Statistical laws in urban mobility from microscopic GPS data in the area of Florence. *Journal of Statistical Mechanics: Theory and Experiment*, 2010(05):P05001, 2010. doi:10.1088/1742-5468/2010/05/P05001.
- [26] Xiao Liang, Jichang Zhao, Li Dong, and Ke Xu. Unraveling the origin of exponential law in intra-urban human mobility. *Scientific Reports*, 3:2983, 2013. doi:10.1038/srep02983.

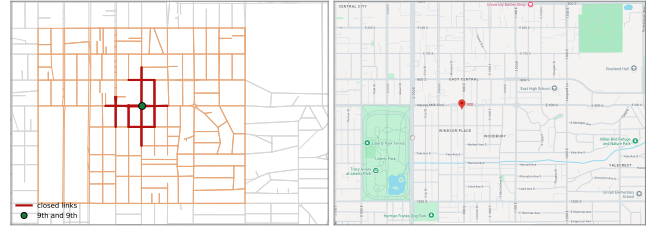


Figure 6: Caption

Table 12: Festival surgery on the expanded closure (14 seed links plus their 1-hop graph neighbors, 42 links). 2018-09-15, 10:00 to 17:55, 96 slots.

region	links	ORIG mean	ORIG median	SURG mean	SURG median	Δ mean
whole network	99,674	1.3266	1.3452	1.3266	1.3452	0.0000
eval box	666	1.4678	1.4910	1.4900	1.5027	+0.0221

Table 13: Festival prior-zero on the expanded closure (14 seed links plus their 1-hop graph neighbors, 42 links). P unchanged, the diffused prior is zeroed on the closed links and re-normalised, the diffused popularity is left untouched. 2018-09-15, 10:00 to 17:55, 96 slots.

region	links	ORIG mean	ORIG median	ZERO mean	ZERO median	Δ mean
whole network	99,716	1.3262	1.3446	1.3262	1.3446	0.0000
eval box	708	1.4860	1.5039	1.3800	1.3951	-0.1060

1277	1335
1278	1336
1279	1337
1280	1338
1281	1339
1282	1340
1283	1341
1284	1342
1285	1343
1286	1344
1287	1345
1288	1346
1289	1347
1290	1348
1291	1349
1292	1350
1293	1351
1294	1352
1295	1353
1296	1354
1297	1355
1298	1356
1299	1357
1300	1358
1301	1359
1302	1360
1303	1361
1304	1362
1305	1363
1306	1364
1307	1365
1308	1366
1309	1367
1310	1368
1311	1369
1312	1370
1313	1371
1314	1372
1315	1373
1316	1374
1317	1375
1318	1376
1319	1377
1320	1378
1321	1379
1322	1380
1323	1381
1324	1382
1325	1383
1326	1384
1327	1385
1328	1386
1329	1387
1330	1388
1331	1389
1332	1390
1333	1391
1334	1392

Table 14: Single-phase origin-informed forecast ($\delta = 0$, $\alpha_{\text{ang}} \geq 0$), trained on the first 23 days, evaluated on the last 7. The fcast columns are the 7-day-ahead forecast (each test slot fed the origins from 7 days earlier).
HAMID: based on what professor said I should keep these table but no need to mention α_{ang} at all

mode	α_s	α_l	α_{ang}	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
uniform	—	—	—	1.1739	0.9544	0.8309	1.2706	1.0545	0.8625	1.2706	1.0545	0.8625
origin	—	—	—	1.1266	0.9311	0.7818	1.2178	1.0277	0.8117	1.2401	1.0411	0.8310
sp.	+2.242	—	—	1.1096	0.9179	0.7466	1.1963	1.0109	0.7759	1.2197	1.0258	0.7960
la.	—	+0.673	—	1.1136	0.9209	0.7586	1.2018	1.0153	0.7884	1.2252	1.0296	0.8086
ang.	—	—	+0.000	1.1266	0.9311	0.7818	1.2178	1.0277	0.8117	—	—	—
sp. + la.	+1.751	+0.436	—	1.1045	0.9138	0.7383	1.1903	1.0057	0.7675	1.2141	1.0211	0.7879
la. + ang.	—	+0.673	+0.000	1.1136	0.9209	0.7586	1.2018	1.0153	0.7885	—	—	—
sp. + ang.	+2.243	—	+0.000	1.1096	0.9179	0.7466	1.1963	1.0109	0.7759	—	—	—
sp. + la. + ang.	+1.893	+0.410	+0.000	1.1046	0.9137	0.7364	1.1902	1.0054	0.7655	—	—	—

Table 15: Two-phase up/down origin-informed forecast (separate $P^\uparrow/P^\downarrow$; $\delta = 0$, $\alpha_{\text{ang}} \geq 0$ shared; $\beta = 0.102$, $\rho = 0.101$), trained on the first 23 days, evaluated on the last 7. The fcast columns are the 7-day-ahead forecast (each test slot fed the origins from 7 days earlier).

mode	$\alpha_s^\uparrow$	$\alpha_s^\downarrow$	$\alpha_l^\uparrow$	$\alpha_l^\downarrow$	α_{ang}	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
uniform	—	—	—	—	—	1.1732	0.9540	0.8309	1.2698	1.0540	0.8625	1.2698	1.0540	0.8625
origin	—	—	—	—	—	1.1211	0.9278	0.7768	1.2117	1.0243	0.8065	1.2365	1.0395	0.8279
sp.	+6.960	−1.518	—	—	+0.000	1.0990	0.9131	0.7446	1.1852	1.0057	0.7739	1.2117	1.0229	0.7966
la.	—	—	+2.320	−0.658	+0.000	1.1034	0.9155	0.7490	1.1902	1.0088	0.7786	1.2176	1.0263	0.8020
ang.	—	—	—	—	+0.000	1.1211	0.9278	0.7768	1.2117	1.0243	0.8065	—	—	—
sp. + la.	+4.649	−0.572	+1.161	−0.167	+0.000	1.0948	0.9091	0.7375	1.1801	1.0009	0.7667	1.2072	1.0189	0.7899
la. + ang.	—	—	+2.319	−0.660	+0.000	1.1034	0.9155	0.7490	1.1902	1.0088	0.7787	—	—	—
sp. + ang.	+7.100	−1.602	—	—	+0.000	1.0990	0.9131	0.7446	1.1852	1.0058	0.7739	—	—	—
sp. + la. + ang.	+4.388	−0.457	+1.184	−0.173	+0.000	1.0948	0.9090	0.7375	1.1802	1.0009	0.7667	—	—	—

1509	1567
1510	1568
1511	1569
1512	1570
1513	1571
1514	1572
1515	1573
1516	1574
1517	1575
1518	1576
1519	1577
1520	1578
1521	1579
1522	1580
1523	1581
1524	1582
1525	1583
1526	1584
1527	1585
1528	1586
1529	1587
1530	1588
1531	1589
1532	1590
1533	1591
1534	1592
1535	1593
1536	1594
1537	1595
1538	1596
1539	1597
1540	1598
1541	1599
1542	1600
1543	1601
1544	1602
1545	1603
1546	1604
1547	1605
1548	1606
1549	1607
1550	1608
1551	1609
1552	1610
1553	1611
1554	1612
1555	1613
1556	1614
1557	1615
1558	1616
1559	1617
1560	1618
1561	1619
1562	1620
1563	1621
1564	1622
1565	1623
1566	1624

Table 16: Single-phase, $\delta = 0.01$, α_{ang} free; HAMID: Based on what the professor said, I must remove these tables. They are absurd because the α_{ang} becomes a negative value.

mode	α_s	α_l	α_{ang}	train mean	train median	train dev	test mean	test median	test dev
uniform	—	—	—	1.1739	0.9544	0.8308	1.2705	1.0544	0.8625
origin	—	—	—	1.1264	0.9310	0.7816	1.2176	1.0276	0.8115
sp.	+2.244	—	+0.000	1.1094	0.9178	0.7464	1.1961	1.0108	0.7757
la.	—	+0.672	+0.000	1.1135	0.9209	0.7585	1.2016	1.0152	0.7884
ang.	—	—	−2.068	1.1116	0.9511	0.6978	1.1930	1.0422	0.7240
sp. + la.	+1.752	+0.435	+0.000	1.1043	0.9138	0.7382	1.1901	1.0056	0.7673
la. + ang.	—	+10.870	−6.602	1.1052	0.9499	0.6849	1.1850	1.0376	0.7106
sp. + ang.	+37.241	—	−7.993	1.0993	0.9447	0.6860	1.1792	1.0329	0.7116
sp. + la. + ang.	+35.397	+1.956	−8.194	1.0981	0.9444	0.6842	1.1777	1.0315	0.7098

Table 17: Two-phase up/down, $\delta = 0.01$, α_{ang} free (shared); $\beta = 0.102$, $\rho = 0.101$.

mode	$\alpha_s^\uparrow$	$\alpha_s^\downarrow$	$\alpha_l^\uparrow$	$\alpha_l^\downarrow$	α_{ang}	train mean	train median	train dev	test mean	test median	test dev
uniform	—	—	—	—	—	1.1732	0.9540	0.8309	1.2698	1.0540	0.8625
origin	—	—	—	—	—	1.1209	0.9278	0.7767	1.2115	1.0242	0.8063
sp.	+6.975	−1.522	—	—	+0.000	1.0989	0.9130	0.7445	1.1851	1.0057	0.7738
la.	—	—	+2.321	−0.658	+0.000	1.1033	0.9154	0.7489	1.1901	1.0087	0.7785
ang.	—	—	—	—	−1.769	1.1057	0.9456	0.6967	1.1871	1.0363	0.7229
sp. + la.	+4.643	−0.568	+1.161	−0.168	+0.000	1.0947	0.9091	0.7374	1.1800	1.0009	0.7666
la. + ang.	—	—	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
sp. + ang.	TBD	TBD	—	—	TBD	TBD	TBD	TBD	TBD	TBD	TBD
sp. + la. + ang.	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 18: Baselines with flipped split: test = first 7 days, train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.2214	0.9967	0.8439	1.1145	0.8895	0.8190
origin	1.1715	0.9715	0.7939	1.0703	0.8670	0.7712
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.2207	0.9962	0.8439	1.1137	0.8890	0.8190
origin	1.1657	0.9684	0.7888	1.0651	0.8642	0.7663

Table 19: Baselines with split: test = second 7 days, train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.1700	0.9507	0.8247	1.2836	1.0582	0.8802
origin	1.1232	0.9263	0.7763	1.2290	1.0304	0.8277
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.1692	0.9502	0.8247	1.2828	1.0577	0.8802
origin	1.1177	0.9235	0.7713	1.2227	1.0271	0.8223

Table 20: Baselines with split: test = third 7 days, train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.1823	0.9489	0.8395	1.2432	1.0355	0.8370
origin	1.1345	0.9265	0.7903	1.1917	1.0050	0.7867
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.1815	0.9484	0.8396	1.2424	1.0350	0.8370
origin	1.1289	0.9237	0.7853	1.1858	1.0005	0.7815

Table 21: Baselines with split: test = day 4–11 (7 days), train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.1809	0.9570	0.8310	1.2478	1.0189	0.8641
origin	1.1334	0.9350	0.7820	1.1954	0.9924	0.8131
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.1801	0.9565	0.8310	1.2470	1.0185	0.8641
origin	1.1279	0.9321	0.7770	1.1893	0.9890	0.8079

Table 22: Baselines with split: test = day 17–24 (7 days), train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.1880	0.9611	0.8377	1.2244	0.9923	0.8441
origin	1.1401	0.9393	0.7886	1.1735	0.9670	0.7934
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.1872	0.9606	0.8377	1.2236	0.9917	0.8441
origin	1.1345	0.9365	0.7836	1.1676	0.9640	0.7882

Table 23: Baselines with split: test = day 12–19 (7 days), train = remaining 23 days.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.1743	0.9469	0.8328	1.2692	1.0460	0.8564
origin	1.1272	0.9249	0.7840	1.2159	1.0170	0.8048
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.1736	0.9464	0.8329	1.2685	1.0455	0.8564
origin	1.1216	0.9217	0.7791	1.2097	1.0138	0.7995

Table 24: Weekend+holiday. Test = last 25% (train = first 75%).

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	0.7215	0.6893	0.4096	0.7188	0.7291	0.4056
origin	0.7042	0.6750	0.3961	0.7017	0.7152	0.3928
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	0.7208	0.6886	0.4098	0.7181	0.7285	0.4059
origin	0.7017	0.6733	0.3949	0.6993	0.7132	0.3916

Table 25: Weekend+holiday. Test = first 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	0.7454	0.7213	0.4176	0.6470	0.6405	0.3708
origin	0.7273	0.7074	0.4037	0.6323	0.6255	0.3597
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	0.7447	0.7207	0.4178	0.6463	0.6398	0.3711
origin	0.7248	0.7055	0.4024	0.6301	0.6235	0.3587

Table 26: Weekend+holiday. Test = second 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	0.7286	0.7157	0.4091	0.6973	0.6336	0.4061
origin	0.7111	0.7008	0.3958	0.6808	0.6191	0.3928
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	0.7279	0.7151	0.4094	0.6966	0.6330	0.4063
origin	0.7087	0.6986	0.3946	0.6785	0.6170	0.3916

Table 27: Weekend+holiday. Test = third 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	0.6877	0.6716	0.3957	0.8201	0.7727	0.4302
origin	0.6716	0.6568	0.3832	0.7994	0.7602	0.4151
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	0.6870	0.6710	0.3959	0.8194	0.7721	0.4304
origin	0.6693	0.6550	0.3821	0.7967	0.7581	0.4137

Table 28: Weekday. Test = last 25% (train = first 75%).

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.4409	1.3901	0.9023	1.5324	1.8750	0.8849
origin	1.3757	1.3485	0.8478	1.4630	1.7951	0.8313
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.4401	1.3897	0.9023	1.5316	1.8743	0.8848
origin	1.3683	1.3446	0.8422	1.4552	1.7856	0.8258

Table 29: Weekday. Test = first 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.4760	1.5008	0.8977	1.4268	1.3960	0.9013
origin	1.4093	1.4542	0.8434	1.3622	1.3536	0.8470
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.4753	1.5003	0.8977	1.4260	1.3956	0.9013
origin	1.4017	1.4485	0.8378	1.3549	1.3476	0.8415

Table 30: Weekday. Test = second 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.4640	1.4792	0.8926	1.4628	1.4560	0.9174
origin	1.3979	1.4319	0.8386	1.3965	1.4088	0.8622
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.4632	1.4787	0.8926	1.4620	1.4555	0.9173
origin	1.3904	1.4264	0.8330	1.3889	1.4018	0.8566

Table 31: Weekday. Test = third 25%.

mode	train mean	train median	train dev	test mean	test median	test dev
<i>Single-phase, $\alpha = 0.0508$</i>						
uniform	1.4740	1.5798	0.9023	1.4329	1.3112	0.8877
origin	1.4072	1.5242	0.8480	1.3683	1.2663	0.8335
<i>Two-phase, $\beta = 0.102, \rho = 0.101$</i>						
uniform	1.4732	1.5794	0.9023	1.4322	1.3107	0.8877
origin	1.3997	1.5183	0.8424	1.3610	1.2618	0.8279

2089	2147
2090	2148
2091	2149
2092	2150
2093	2151
2094	2152
2095	2153
2096	2154
2097	2155
2098	2156
2099	2157
2100	2158
2101	2159
2102	2160
2103	2161
2104	2162
2105	2163
2106	2164
2107	2165
2108	2166
2109	2167
2110	2168
2111	2169
2112	2170
2113	2171
2114	2172
2115	2173
2116	2174
2117	2175
2118	2176
2119	2177
2120	2178
2121	2179
2122	2180
2123	2181
2124	2182
2125	2183
2126	2184
2127	2185
2128	2186
2129	2187
2130	2188
2131	2189
2132	2190
2133	2191
2134	2192
2135	2193
2136	2194
2137	2195
2138	2196
2139	2197
2140	2198
2141	2199
2142	2200
2143	2201
2144	2202
2145	2203
2146	2204

2205	2263
2206	2264
2207	2265
2208	2266
2209	2267
2210	2268
2211	2269
2212	2270
2213	2271
2214	2272
2215	2273
2216	2274
2217	2275
2218	2276
2219	2277
2220	2278
2221	2279
2222	2280
2223	2281
2224	2282
2225	2283
2226	2284
2227	2285
2228	2286
2229	2287
2230	2288
2231	2289
2232	2290
2233	2291
2234	2292
2235	2293
2236	2294
2237	2295
2238	2296
2239	2297
2240	2298
2241	2299
2242	2300
2243	2301
2244	2302
2245	2303
2246	2304
2247	2305
2248	2306
2249	2307
2250	2308
2251	2309
2252	2310
2253	2311
2254	2312
2255	2313
2256	2314
2257	2315
2258	2316
2259	2317
2260	2318
2261	2319
2262	2320

Table 32: Single-phase sp. +1a. trained per day-category ($\alpha = 0.0508$; test = second 25% of the category). fcast = 7-day-ahead forecast (weekend covers 690 of 777 test slots; the earliest 87 lack origins 7 days earlier).

category	α_s	α_l	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
weekend+holiday	+0.844	+0.290	0.7050	0.6937	0.3871	0.6752	0.6138	0.3843	0.6827	0.5583	0.4093
weekday	+2.523	+0.576	1.3568	1.3812	0.7648	1.3568	1.3561	0.7860	1.3893	1.3847	0.8113

Table 33: Two-phase up/down sp. +1a. trained per day-category ($\beta = 0.102$, $\rho = 0.101$; test = second 25% of the category). fcast = 7-day-ahead forecast (weekend covers 690 of 777 test slots; the earliest 87 lack origins 7 days earlier).

category	$\alpha_s^\uparrow$	$\alpha_s^\downarrow$	$\alpha_l^\uparrow$	$\alpha_l^\downarrow$	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
weekend+holiday	+3.359	-1.158	+0.815	-0.281	0.6998	0.6890	0.3842	0.6702	0.6103	0.3814	0.6793	0.5563	0.4075
weekday	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 34: Single-phase sp. +1a. trained per day of week ($\alpha = 0.0508$). Test = that weekday in the second 7-day window; train = its other occurrences; fcast = 7-day-ahead forecast (same weekday's week-1 origins fed to predict the test day).

day	α_s	α_l	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
Monday	+1.673	+0.410	1.1137	0.9161	0.7723	1.3646	1.5670	0.8323	1.4251	1.6162	0.8836
Tuesday	+2.363	+0.537	1.3223	1.2622	0.7764	1.3880	1.6137	0.8031	1.4189	1.6514	0.8256
Wednesday	+2.526	+0.580	1.3578	1.3803	0.7601	1.3942	1.6047	0.7855	1.4265	1.6373	0.8096
Thursday	+2.558	+0.580	1.3624	1.3840	0.7699	1.4099	1.6065	0.7820	1.4410	1.6480	0.8049
Friday	+2.557	+0.598	1.3366	1.3031	0.7366	1.3891	1.5171	0.7715	1.4200	1.5526	0.7943
Saturday	+1.158	+0.369	0.8133	0.8084	0.4186	0.8604	0.9102	0.4382	0.8763	0.9253	0.4497
Sunday	+0.607	+0.239	0.5825	0.6034	0.3042	0.5730	0.4974	0.3113	0.5819	0.5058	0.3180

Table 35: Two-phase up/down sp. +1a. trained per day of week ($\beta = 0.102$, $\rho = 0.101$; test = that weekday in the second 7-day window, train = its other occurrences). fcast = 7-day-ahead forecast (that weekday's week-1 origins fed to predict the test day).

day	$\alpha_s^\uparrow$	$\alpha_s^\downarrow$	$\alpha_l^\uparrow$	$\alpha_l^\downarrow$	train mean	train median	train dev	test mean	test median	test dev	fcast mean	fcast median	fcast dev
Monday	+4.415	-0.714	+1.111	-0.213	1.1037	0.9124	0.7719	1.3535	1.5608	0.8316	1.4205	1.6155	0.8883
Tuesday	+5.014	+0.101	+1.376	-0.094	1.3126	1.2599	0.7843	1.3783	1.6135	0.8113	1.4132	1.6558	0.8367
Wednesday	+5.189	+0.364	+1.479	-0.057	1.3485	1.3781	0.7687	1.3848	1.6031	0.7941	1.4213	1.6403	0.8213
Thursday	+5.192	+0.414	+1.482	-0.054	1.3531	1.3808	0.7786	1.4008	1.6057	0.7905	1.4359	1.6516	0.8164
Friday	+5.471	+0.388	+1.533	-0.021	1.3274	1.3004	0.7439	1.3799	1.5164	0.7788	1.4149	1.5567	0.8045
Saturday	+4.024	-0.906	+1.022	-0.243	0.8070	0.8045	0.4163	0.8536	0.9055	0.4359	0.8718	0.9216	0.4489
Sunday	+2.883	-1.219	+0.643	-0.260	0.5786	0.5995	0.3020	0.5693	0.4940	0.3088	0.5794	0.5039	0.3164